

APOSTILA COMPLETA

App Voll em React Native com Expo

Guia passo a passo do desenvolvimento completo: componentes, navegação, validação de formulários, máscaras de input e fluxo de dados entre telas.

React Native 0.73

Expo 50

React Navigation 6

react-native-mask-input

react-native-svg

1 O que é este projeto?

O **App Voll** é um aplicativo mobile criado com **React Native + Expo**. Ele simula o fluxo de **login** e **cadastro** de um aplicativo de saúde, onde o usuário:

- 1 Faz login com e-mail e senha.
- 2 Caso não tenha conta, realiza um cadastro dividido em **3 passos**.
- 3 **Passo 1** — dados pessoais: nome, e-mail, senha.
- 4 **Passo 2** — endereço: CEP, logradouro, número, complemento, telefone.
- 5 **Passo 3** — plano de saúde: seleção por checkboxes.



Por que dividir o cadastro em etapas?

Formulários longos em uma única tela aumentam a taxa de abandono. Dividir em passos melhora a experiência do usuário (UX) e permite validar cada grupo de campos antes de avançar.

2 Tecnologias e bibliotecas utilizadas

Biblioteca	Versão	Para que serve
<code>expo</code>	~50.0.17	Plataforma que facilita o desenvolvimento React Native
<code>react</code>	18.2.0	Biblioteca principal para criar componentes
<code>react-native</code>	0.73.6	Framework para criar apps móveis com JavaScript
<code>@react-navigation/native</code>	^6.1.17	Sistema de navegação entre telas
<code>@react-navigation/native-stack</code>	^6.9.26	Navegação em pilha (stack) com animações nativas
<code>react-native-safe-area-context</code>	4.8.2	Respeita notch, barra de status e áreas seguras
<code>react-native-screens</code>	~3.29.0	Otimiza renderização usando telas nativas do SO
<code>react-native-mask-input</code>	^1.2.3	Formata inputs com máscaras (CEP, telefone)
<code>react-native-svg</code>	14.1.0	Permite desenhar vetores SVG (logo)
<code>expo-status-bar</code>	~1.11.1	Controla a barra de status do celular

Como instalar e rodar

```
# Instala todas as dependências do package.json
npm install

# Inicia o servidor de desenvolvimento
npx expo start
```

```
# Para rodar diretamente em um dispositivo/emulador  
npx expo start --android  
npx expo start --ios
```

3 Estrutura de pastas

```
app_voll/  
├─ index.js           ← Ponto de entrada do Expo  
├─ App.jsx            ← Componente raiz da aplicação  
├─ package.json       ← Lista de dependências e scripts  
├─ babel.config.js    ← Configuração do compilador  
├─ app.json           ← Configurações do Expo (nome, ícone, etc.)  
├─  
└─ src/  
    ├─ components/  
    │   ├─ Button.jsx ← Botão primário e secundário  
    │   ├─ Checkbox.jsx ← Caixa de seleção com rótulo  
    │   ├─ Input.jsx ← Campo de texto com suporte a máscara  
    │   ├─ Logo.jsx ← Logo SVG do app  
    │   └─ ScreenContainer.jsx ← Wrapper padrão de cada tela  
    ├─ navigation/  
    │   └─ AppNavigator.jsx ← Define as rotas e como navegar entre elas  
    ├─ screens/  
    │   └─ Cada arquivo é uma tela completa  
    │       ├─ LoginScreen.jsx  
    │       ├─ CadastroPasso1Screen.jsx  
    │       ├─ CadastroPasso2Screen.jsx  
    │       └─ CadastroPasso3Screen.jsx  
    ├─ theme/  
    │   └─ colors.js ← Paleta de cores centralizada  
    └─ utils/  
        └─ masks.js ← Máscaras e funções de validação
```



Por que essa estrutura?

Separar em `components`, `screens`, `navigation`, `theme` e `utils` é uma convenção amplamente usada em React Native. Facilita encontrar arquivos, reutilizar código e escalar o projeto.

4 Como o projeto inicia — index.js e App.jsx

index.js

```
import { registerRootComponent } from 'expo'; // Importa a função que registra o
import App from './App';                      // Importa o componente principal

registerRootComponent(App); // Diz ao Expo: "este é o componente inicial, renderi
```

App.jsx

```
import React from 'react'; // Importa o React – nec
import { StatusBar } from 'expo-status-bar'; // Controla a barra de s
import { SafeAreaProvider } from 'react-native-safe-area-context'; // Provê conte
import AppNavigator from './src/navigation/AppNavigator'; // Importa o navegador

export default function App() { // Declara e exporta o c
  return (
    <SafeAreaProvider> // Envolve todo o app pa
                        // saibam onde ficam o
    <StatusBar style="auto" /> // Ajusta a barra de sta
    <AppNavigator /> // Renderiza o sistema d
  </SafeAreaProvider>
  );
}
```



SafeAreaProvider

— Em celulares modernos existem "áreas perigosas" onde a interface do sistema fica (notch, câmera de buraco, barra de gestos). O `SafeAreaProvider` calcula essas áreas e disponibiliza a informação para todos os componentes filhos via contexto React. Sem ele, o conteúdo pode ficar escondido atrás do notch.

5

Sistema de cores — theme/colors.js

```
// Paleta de cores do app Voll
const colors = {
  primary:      '#0B3B60', // Azul escuro – botões principais, bordas de inp
  primaryDark:  '#082846', // Azul ainda mais escuro – reservado para variaç
  accent:       '#2196F3', // Azul claro – logo, links e destaques.
  textPrimary:  '#333333', // Cor padrão de texto – quase preto, mais suave
  textSecondary: '#6B7280', // Cinza médio – textos secundários, subtítulos.
  inputBackground: '#F4F4F4', // Fundo levemente acinzentado dos campos de text
  inputBorder:   '#E5E7EB', // Borda dos campos de texto em estado normal.
  placeholder:   '#9CA3AF', // Cor do texto de placeholder (dica dentro do ca
  white:         '#FFFFFF', // Branco – texto em botões e elementos no fundo
  disabled:      '#A3A3A3', // Cinza – fundo do botão "Voltar" (variante secu
  backgroundDark: '#000000', // Preto – fundo da tela 3 (seleção de planos).
  error:         '#DC2626', // Vermelho – bordas e mensagens de erro.
};

export default colors; // Exporta o objeto para que outros arquivos possam import
```



Princípio DRY

— Imagine que a cor primária mude de azul para verde. Sem centralização, você editaria dezenas de arquivos. Com um arquivo de tema, muda apenas uma linha e todo o app atualiza.

6

Utilitários de máscara e validação — utils/masks.js

```
// Máscara de CEP: formata "12345678" como "12345-678"
export const CEP_MASK = [
  /\d/, // 1° dígito: aceita qualquer número (\d = 0-9)
  /\d/, // 2° dígito
  /\d/, // 3° dígito
  /\d/, // 4° dígito
  /\d/, // 5° dígito
  '-', // Caractere fixo: o hífen é inserido automaticamente
  /\d/, // 6° dígito
  /\d/, // 7° dígito
  /\d/, // 8° dígito
];

// Máscara de telefone: formata "11912345678" como "(11) 91234-5678"
export const PHONE_MASK = [
  '(', // Parêntese de abertura (inserido automaticamente)
  /\d/, // DDD - 1° dígito
  /\d/, // DDD - 2° dígito
  ')', // Parêntese de fechamento
  ' ', // Espaço fixo
  /\d/, /\d/, /\d/, /\d/, /\d/, // 5 dígitos do número
  '-', // Hífen separador
  /\d/, /\d/, /\d/, /\d/, // 4 dígitos finais
];

// Remove tudo que não é dígito – útil para validar sem a formatação
export const onlyDigits = (value) =>
  (value || '').replace(/\D/g, '');
// (value || '') → se value for null/undefined, usa string vazia
// .replace(/\D/g, '') → apaga todos os não-dígitos (\D)
// A flag 'g' = "global": substitui TODAS as ocorrências
```



```
// Validação básica de e-mail
export const isValidEmail = (email) => {
  if (!email) return false; // Vazio → inválido
  const re = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
  // ^ → início | [^\s@]+ → chars antes do @ | @ → arroba
  // [^\s@]+ → domínio | \. → ponto literal | [^\s@]+$ → extensão até o fim
  return re.test(email.trim()); // .trim() remove espaços das bordas
};

// CEP deve ter exatamente 8 dígitos (após remover o hífen)
export const isValidCEP = (cep) => onlyDigits(cep).length === 8;
// Exemplo: "12345-678" → onlyDigits → "12345678" → length 8 → true

// Telefone: 10 dígitos (fixo) ou 11 dígitos (celular com 9°)
export const isValidPhone = (phone) => {
  const d = onlyDigits(phone).length;
  return d === 10 || d === 11;
};
```



Como a máscara funciona?

Cada elemento do array é aplicado posição a posição ao texto digitado. Se o elemento é um `RegExp` (ex: `/\d/`), o caractere digitado precisa satisfazê-lo. Se é uma string (ex: `'-'`), é inserida automaticamente sem que o usuário precise digitar.

7.1 Componente — ScreenContainer.jsx

Este é o **invólucro (wrapper) padrão** de todas as telas. Ele resolve três problemas comuns:

Problema	Solução
Conteúdo escondido atrás do notch/barra de status	SafeAreaView
Teclado sobrepõe os campos quando abre	KeyboardAvoidingView
Conteúdo maior que a tela não rola	ScrollView

```
import React from 'react';
import { View, ScrollView, KeyboardAvoidingView, Platform, StyleSheet } from 'react-native';
import { SafeAreaView } from 'react-native-safe-area-context';
import colors from '../theme/colors';

export default function ScreenContainer({ children, dark = false }) {
  return (
    <SafeAreaView
      style={[styles.safe, dark && styles.safeDark]}
      // Array de estilos: sempre aplica 'safe', adiciona 'safeDark' SOMENTE se dark for true
      edges={['top', 'left', 'right', 'bottom']}
      // Respeita as áreas seguras em todas as bordas
    >
      <KeyboardAvoidingView
        style={styles.flex}
        behavior={Platform.OS === 'ios' ? 'padding' : undefined}
        // iOS: adiciona padding embaixo do conteúdo quando o teclado abre
        // Android: o SO já faz esse ajuste automaticamente → undefined
      >
        <ScrollView
          contentContainerStyle={styles.scroll}
          keyboardShouldPersistTaps="handled"
          // "handled": toque fora do teclado fecha-o E processa o toque (ex: cli
```

```
      showsVerticalScrollIndicator={false} // Oculta a barra de rolagem lateral
    >
    <View style={styles.inner}>{children}</View>
  </ScrollView>
</KeyboardAvoidingView>
</SafeAreaView>
);
}

const styles = StyleSheet.create({
  safe: { flex: 1, backgroundColor: colors.white }, // Fundo branco padrão
  safeDark: { backgroundColor: colors.backgroundDark }, // Fundo preto quando
  flex: { flex: 1 },
  scroll: { flexGrow: 1, paddingHorizontal: 24, paddingTop: 16, paddingBottom: 32 },
  inner: { flex: 1 },
});
```

7.2 Componente — Logo.jsx

```
import React from 'react';
import { View, Text, StyleSheet } from 'react-native';
import Svg, { Path } from 'react-native-svg'; // Primitivos de SVG
import colors from '../theme/colors';

export default function Logo({ size = 28 }) { // size padrão: 28px
  const iconSize = size;
  const fontSize = size * 1.15; // Texto 15% maior que o ícone

  return (
    <View style={styles.wrapper}> // Container em linha (row)
      <Svg width={iconSize} height={iconSize} viewBox="0 0 48 48">
        // viewBox define as coordenadas internas do SVG (de 0 a 48)

        // Estrela nos eixos horizontal e vertical
        <Path
          d="M24 4 L26 22 L44 24 L26 26 L24 44 L22 26 L4 24 L22 22 Z"
          // M24 4 → Começa no topo | L44 24 → ponta direita | L4 24 → ponta esquerda
          fill={colors.accent}
        />

        // Estrela rotacionada 45° (pontas diagonais)
        <Path
          d="M10 10 L24 22 L38 10 L26 24 L38 38 L24 26 L10 38 L22 24 Z"
          fill={colors.accent}
          opacity={0.85} // Levemente transparente para criar profundidade
        />
      </Svg>

      <Text style={[styles.text, { fontSize }]}>voll</Text>
    </View>
  );
}
```

```
const styles = StyleSheet.create({  
  wrapper: { flexDirection: 'row', alignItems: 'center', justifyContent: 'center'  
  text: { color: colors.accent, fontWeight: '600', letterSpacing: 0.5 },  
});
```

7.3 Componente — Input.jsx

O `Input` é o componente mais complexo do projeto. Ele suporta dois modos: **campo normal** e **campo com máscara**.

```
import React from 'react';
import { View, Text, TextInput, StyleSheet } from 'react-native';
import MaskInput from 'react-native-mask-input';
import colors from '../theme/colors';

export default function Input({
  label, // Texto exibido acima do campo
  placeholder, // Dica dentro do campo vazio
  value, // Valor atual – controlado pelo pai
  onChangeText, // Função chamada a cada digitação
  secureTextEntry, // true → oculta texto (campo de senha)
  keyboardType = 'default', // Tipo de teclado a abrir
  autoCapitalize = 'sentences', // Capitalização automática
  mask, // Array de máscara – opcional
  error, // Mensagem de erro – exibe borda vermelha
  onDark = false, // true → cores para fundo escuro
}) {
  // Escolha dinâmica: MaskInput quando há máscara, TextInput quando não
  const Field = mask ? MaskInput : TextInput;

  const fieldProps = mask
    ? {
      mask,
      value,
      onChangeText: (masked, unmasked) => onChangeText(masked, unmasked),
      // MaskInput passa (masked, unmasked) – o valor formatado e o valor cru
    }
    : { value, onChangeText }; // TextInput passa apenas o valor
```

```

return (
  <View style={styles.container}>
    {label ? (
      <Text style={[styles.label, onDark && styles.labelDark]}>{label}</Text>
      // labelDark: usa cor de destaque (azul claro) em fundo preto
    ) : null}

    <Field
      {...fieldProps} // Espalha as props preparadas
      placeholder={placeholder}
      placeholderTextColor={colors.placeholder}
      secureTextEntry={secureTextEntry}
      keyboardType={keyboardType}
      autoCapitalize={autoCapitalize}
      style={[styles.input, error && styles.inputError]}
      // inputError: borda vermelha quando há mensagem de erro
    />

    {error ? <Text style={styles.errorText}>{error}</Text> : null}
    // Exibe mensagem de erro somente quando "error" tem valor
  </View>
);
}

const styles = StyleSheet.create({
  container: { marginBottom: 16 }, // Espaço entre campos
  label: { color: colors.primary, fontSize: 15, fontWeight: '700', marginBot
  labelDark: { color: colors.accent }, // Azul claro em fundo e
  input: {
    backgroundColor: colors.inputBackground, // Cinza claro (#F4F4F4
    borderRadius: 10, paddingHorizontal: 14, paddingVertical: 12,
    fontSize: 15, color: colors.textPrimary,
    borderWidth: 1, borderColor: colors.inputBorder, // Borda padrão
  },
  inputError: { borderColor: colors.error }, // Troca borda para verme
  errorText: { color: colors.error, fontSize: 12, marginTop: 4 },
});

```



Componente Controlado

— O `Input` não guarda o texto em si. O valor vem sempre do estado do pai via `value`, e cada digitação chama `onChangeText` para que o pai atualize seu estado. Isso dá controle total ao componente pai sobre os dados do formulário.

7.4 Componente — Button.jsx

```
import React from 'react';
import { TouchableOpacity, Text, StyleSheet, ActivityIndicator } from 'react-native';
import colors from '../theme/colors';

export default function Button({
  title, // Texto do botão
  onPress, // Função chamada ao pressionar
  variant = 'primary', // 'primary' (azul) ou 'secondary' (cinza) – padrão: primary
  disabled = false, // Se true, desabilita o toque
  loading = false, // Se true, exibe spinner em vez do texto
  style, // Estilos extras do componente pai
}) {
  const isSecondary = variant === 'secondary';

  return (
    <TouchableOpacity
      activeOpacity={0.85} // 85% de opacidade ao pressionar (feedback visual)
      onPress={onPress}
      disabled={disabled || loading} // Desabilitado se marcado ou carregando
      style={[
        styles.base,
        isSecondary ? styles.secondary : styles.primary, // Cor conforme variante
        (disabled || loading) && styles.disabled, // Reduz opacidade quando desabilitado ou carregando
        style, // Estilos extras do pai
      ]}
    >
      {loading
        ? <ActivityIndicator color={colors.white} /> // Spinner durante carregamento
        : <Text style={styles.text}>{title}</Text>
      }
    </TouchableOpacity>
  );
}
```

```
}

const styles = StyleSheet.create({
  base:      { paddingVertical: 14, borderRadius: 10, alignItems: 'center', justi
  primary:   { backgroundColor: colors.primary },    // Azul escuro (#0B3B60)
  secondary: { backgroundColor: colors.disabled },    // Cinza (#A3A3A3)
  disabled:  { opacity: 0.7 },                      // Botão inativo = semi-tran
  text:      { color: colors.white, fontSize: 16, fontWeight: '700' },
});
```

7.5 Componente — Checkbox.jsx

```
import React from 'react';
import { TouchableOpacity, Text, View, StyleSheet } from 'react-native';
import colors from '../theme/colors';

export default function Checkbox({ label, checked, onPress, onDark = false }) {
  return (
    <TouchableOpacity
      style={styles.row}
      onPress={onPress}      // Notifica o pai para alternar o estado
      activeOpacity={0.7}    // Feedback visual ao pressionar
    >
      <View style={[
        styles.box,
        onDark && styles.boxDark,      // Borda branca em fundo escuro
        checked && styles.boxChecked,  // Fundo azul quando marcado
      ]>
        {checked ? <Text style={styles.check}>✓</Text> : null}
        // Exibe o ✓ somente quando checked=true
      </View>

      <Text style={[styles.label, onDark && styles.labelDark]}>{label}</Text>
    </TouchableOpacity>
  );
}

const styles = StyleSheet.create({
  row: { flexDirection: 'row', alignItems: 'center', paddingVertical: 8 },
  box: {
    width: 22, height: 22, borderRadius: 4,
    borderWidth: 1.5, borderColor: colors.primary,
    marginRight: 12,                                // Espaço entre caixa e t
    alignItems: 'center', justifyContent: 'center',
```

```
    backgroundColor: 'transparent', // Transparente quando des
  },
  boxDark: { borderColor: colors.white }, // Borda branca em fundo
  boxChecked: { backgroundColor: colors.primary, borderColor: colors.primary }, /
  check: { color: colors.white, fontSize: 14, fontWeight: '700', lineHeight:
  label: { fontSize: 16, color: colors.textPrimary },
  labelDark: { color: colors.white },
});
```

8 Navegação — AppNavigator.jsx

```
import React from 'react';
import { NavigationContainer } from '@react-navigation/native';
// NavigationContainer: contexto global de navegação. Mantém o histórico de telas
import { createNativeStackNavigator } from '@react-navigation/native-stack';
// createNativeStackNavigator: navegador de pilha usando componentes nativos do i

import LoginScreen from '../screens/LoginScreen';
import CadastroPasso1Screen from '../screens/CadastroPasso1Screen';
import CadastroPasso2Screen from '../screens/CadastroPasso2Screen';
import CadastroPasso3Screen from '../screens/CadastroPasso3Screen';

const Stack = createNativeStackNavigator(); // Cria a instância do navegador

export default function AppNavigator() {
  return (
    <NavigationContainer>
      <Stack.Navigator
        initialRouteName="Login" // Tela exibida ao abrir o app
        screenOptions={{
          headerShown: false, // Oculta o cabeçalho padrão em TODAS as
          animation: 'slide_from_right', // Animação: desliza da direita para a e
        }}
      >
        <Stack.Screen name="Login" component={LoginScreen} />
        <Stack.Screen name="CadastroPasso1" component={CadastroPasso1Screen} />
        <Stack.Screen name="CadastroPasso2" component={CadastroPasso2Screen} />
        <Stack.Screen name="CadastroPasso3" component={CadastroPasso3Screen} />
      </Stack.Navigator>
    </NavigationContainer>
  );
}
```

Como funciona a pilha de navegação?

Estado inicial:

Login

→

Após navegar:

Passo 3
Passo 2
Passo 1
Login

→

Após popToTop():

Login

Método	O que faz
<code>navigation.navigate('Nome', {params})</code>	Vai para uma tela, passando dados opcionais
<code>navigation.goBack()</code>	Volta para a tela anterior (remove o topo da pilha)
<code>navigation.popToTop()</code>	Volta para a primeira tela (esvazia toda a pilha)

9.1 Tela — LoginScreen.jsx

```
import React, { useState } from 'react';
import { View, Text, StyleSheet, TouchableOpacity, Alert } from 'react-native';
import ScreenContainer from '../components/ScreenContainer';
import Logo from '../components/Logo';
import Input from '../components/Input';
import Button from '../components/Button';
import colors from '../theme/colors';
import { isValidEmail } from '../utils/masks';

// "navigation" é injetado automaticamente pelo React Navigation
export default function LoginScreen({ navigation }) {

  // Estado de cada campo do formulário
  const [email, setEmail] = useState(''); // Campo de email (começa vazio)
  const [senha, setSenha] = useState(''); // Campo de senha (começa vazio)
  const [errors, setErrors] = useState({}); // Objeto com erros por campo

  const handleLogin = () => {
    const newErrors = {}; // Objeto vazio a cada nova tentativa

    if (!email.trim()) newErrors.email = 'Informe seu email';
    else if (!isValidEmail(email)) newErrors.email = 'Email inválido';
    // .trim() remove espaços das extremidades antes de checar se está vazio

    if (!senha) newErrors.senha = 'Informe sua senha';

    setErrors(newErrors); // Atualiza erros → Input exibe as mensagens

    if (Object.keys(newErrors).length === 0) {
      // Object.keys retorna as chaves do objeto
      // Length === 0 → sem erros → pode prosseguir
      // ↓ Ponto de integração: chamar API de login aqui
    }
  }
}
```

```
Alert.alert('Login', 'Login realizado com sucesso!');
}
};

return (
  <ScreenContainer>
    <View style={styles.logoWrap}>
      <Logo size={36} /> // Logo maior na tela principal
    </View>

    <Text style={styles.title}>Faça login em sua conta</Text>

    <View style={styles.form}>
      <Input
        label="Email"
        placeholder="Insira seu endereço de email"
        value={email}
        onChangeText={(v) => setEmail(v)}
        keyboardType="email-address" // Teclado com @ em destaque
        autoCapitalize="none" // Sem capitalização em emails
        error={errors.email}
      />
      <Input
        label="Senha"
        value={senha}
        onChangeText={setSenha} // Atalho: passa a função setState
        secureTextEntry // Oculta os caracteres digitados
        error={errors.senha}
      />
      <Button title="Entrar" onPress={handleLogin} />

      <TouchableOpacity style={styles.linkWrap}
        onPress={() => Alert.alert('Recuperar senha', 'Fluxo de recuperação.')}
      >
        <Text style={styles.linkUnderline}>Esqueceu sua senha?</Text>
      </TouchableOpacity>
    </View>

    <View style={styles.footer}>
```



```
<Text style={styles.footerText}>
  Ainda não tem conta?{' '}          // {' '} insere espaço em branco no J
  <Text
    style={styles.footerLink}
    onPress={() => navigation.navigate('CadastroPasso1')}
    // Navega sem parâmetros → Passo1 inicia com campos vazios
  >
    Faça seu cadastro!
  </Text>
</Text>
</View>
</ScreenContainer>
);
}
```

9.2 Tela — CadastroPasso1Screen.jsx

```
export default function CadastroPasso1Screen({ navigation, route }) {  
  
  // Recupera dados anteriores se o usuário voltou desta tela  
  // route?.params?.dados → acesso seguro com optional chaining  
  // || {} → usa objeto vazio se não houver dados anteriores  
  const prev = route?.params?.dados || {};  
  
  // Cada campo inicia com o valor anterior (se voltar) ou vazio (primeiro acesso)  
  const [nome, setNome] = useState(prev.nome || '');  
  const [email, setEmail] = useState(prev.email || '');  
  const [senha, setSenha] = useState(prev.senha || '');  
  const [repetirSenha, setRepetirSenha] = useState(prev.repetirSenha || '');  
  const [errors, setErrors] = useState({});  
  
  const validar = () => {  
    const e = {};  
  
    if (!nome.trim())  
      e.nome = 'Informe seu nome';  
    else if (nome.trim().split(' ').length < 2)  
      e.nome = 'Digite seu nome completo';  
    // .split(' ') → ['João'] (1 palavra) ou ['João', 'Silva'] (2 palavras)  
    // length < 2 → exige pelo menos 2 palavras (nome completo)  
  
    if (!email.trim()) e.email = 'Informe seu email';  
    else if (!isValidEmail(email)) e.email = 'Email inválido';  
  
    if (!senha) e.senha = 'Informe uma senha';  
    else if (senha.length < 6) e.senha = 'A senha deve ter pelo menos 6 caractere';  
  
    if (!repetirSenha) e.repetirSenha = 'Confirme a senha';  
    else if (repetirSenha !== senha) e.repetirSenha = 'As senhas não coincidem';  
  }  
}
```

```
// !== compara valor E tipo (comparação estrita)

setErrors(e);
return Object.keys(e).length === 0; // true se não há erros
};

const handleAvancar = () => {
  if (!validar()) return; // Interrompe se houver erro

  navigation.navigate('CadastroPasso2', {
    dados: { nome, email, senha, repetirSenha },
    // Passo2 receberá isso em route.params.dados
  });
};

// ... JSX com 4 Inputs e 1 Button "Avançar"
}
```

9.3 Tela — CadastroPasso2Screen.jsx

```
import { CEP_MASK, PHONE_MASK, isValidCEP, isValidPhone } from '../utils/masks';

export default function CadastroPasso2Screen({ navigation, route }) {
  const dadosPasso1 = route?.params?.dados || {}; // Dados do Passo 1 – repassa
  const prev          = route?.params?.endereco || {}; // Dados de endereço se o us

  const [cep, setCep] = useState(prev.cep || '');
  const [endereco, setEndereco] = useState(prev.endereco || '');
  const [numero, setNumero] = useState(prev.numero || '');
  const [complemento, setComplemento] = useState(prev.complemento || '');
  // complemento é opcional – não tem validação obrigatória
  const [telefone, setTelefone] = useState(prev.telefone || '');
  const [errors, setErrors] = useState({});

  const validar = () => {
    const e = {};
    if (!cep.trim()) e.cep = 'Informe seu CEP';
    else if (!isValidCEP(cep)) e.cep = 'CEP inválido';
    // isValidCEP: remove a máscara e confere se há 8 dígitos

    if (!endereco.trim()) e.endereco = 'Informe seu endereço';
    if (!numero.trim()) e.numero = 'Informe o número';
    // complemento → sem validação (campo opcional)

    if (!telefone.trim()) e.telefone = 'Informe seu telefone';
    else if (!isValidPhone(telefone)) e.telefone = 'Telefone inválido';

    setErrors(e);
    return Object.keys(e).length === 0;
  };

  const handleAvancar = () => {
```

```
if (!validar()) return;
navigation.navigate('CadastroPasso3', {
  dados: dadosPasso1, // Repassa dados do Passo
  endereco: { cep, endereco, numero, complemento, telefone }, // Novos dados
});

// No JSX: dois campos com máscara (CEP e telefone) e dois botões (Voltar + Avançar)
<Input
  label="CEP"
  value={cep}
  onChangeText={({masked} => setCep(masked)) // Usa só o valor mascarado
  mask={CEP_MASK} // Formata como "00000-000"
  keyboardType="numeric" // Teclado numérico
  error={errors.cep}
/>
}
```

9.4 Tela — CadastroPasso3Screen.jsx

```
// Lista fixa fora do componente – não é recriada a cada render
const PLANOS = [
  'Sulamerica', 'Unimed', 'Bradesco', 'Amil',
  'Biosaúde', 'Biovida', 'Outros', 'Não tenho plano',
];

export default function CadastroPasso3Screen({ navigation, route }) {
  const dadosPasso1 = route?.params?.dados || {}; // nome, email, senha, repe
  const endereco = route?.params?.endereco || {}; // cep, logradouro, numero,

  const [selecionados, setSelecionados] = useState([]); // Array de nomes selecio
  const [erro, setErro] = useState(''); // Erro global (string sim

  // Lógica de toggle: marcar/desmarcar um plano
  const toggle = (plano) => {
    setErro(''); // Limpa erro ao interagir

    // "Não tenho plano" é exclusivo: deseleciona todos os outros
    if (plano === 'Não tenho plano') {
      setSelecionados(selecionados.includes(plano) ? [] : [plano]);
      // Se já marcado → desmarca (array vazio) | Se não marcado → seleciona só e
      return; // Não executa o código abaixo
    }

    // Para outros planos: remove "Não tenho plano" se estava marcado
    const semSemPlano = selecionados.filter((p) => p !== 'Não tenho plano');
    // .filter cria novo array mantendo só os que passam no teste

    if (semSemPlano.includes(plano)) {
      setSelecionados(semSemPlano.filter((p) => p !== plano)); // DESMARCA
    } else {
      setSelecionados([...semSemPlano, plano]); // MARCA – spread cria novo array
```

```
    }  
  };  
  
  const handleCadastrar = () => {  
    if (selecionados.length === 0) {  
      setErro('Selecione pelo menos uma opção');  
      return;  
    }  
  
    // Monta o payload final com TODOS os dados das 3 telas  
    const payload = {  
      ...dadosPasso1,      // Espalha: nome, email, senha, repetirSenha  
      endereco,            // Objeto completo de endereço  
      planos: selecionados, // Array dos planos escolhidos  
    };  
  
    // ↓ Ponto de integração: chamar API aqui com o payload  
    console.log('Cadastro enviado:', payload);  
  
    Alert.alert('Cadastro', 'Cadastro realizado com sucesso!', [  
      { text: 'OK', onPress: () => navigation.popToTop() },  
      // popToTop: remove todas as telas da pilha exceto a primeira (Login)  
    ]);  
  };  
  
  return (  
    <ScreenContainer dark>      // Fundo preto na tela de planos  
      ...  
      // Lista de checkboxes gerada pelo .map() sobre o array PLANOS  
      {PLANOS.map((plano) => (  
        <Checkbox  
          key={plano}              // Identificador único obrigatório  
          label={plano}  
          checked={selecionados.includes(plano)} // true se está no array  
          onPress={() => toggle(plano)}  
          onDark                  // Estilo para fundo escuro  
        />  
      )}}  
    </ScreenContainer>  
  );
```

```
);  
}
```


10 Fluxo de dados entre telas



```
navigation.popToTop() → volta para o Login
```



Por que não usar estado global (Redux, Context)?

Neste projeto, a transferência via `route.params` é suficiente porque o fluxo é linear e os dados só importam durante o cadastro. Para apps maiores, onde múltiplas telas não relacionadas precisam dos mesmos dados, usaríamos Context API ou Redux.

11 Conceitos fundamentais utilizados

useState — Gerenciamento de estado local

```
const [valor, setValor] = useState('inicial');  
// valor → lê o estado atual  
// setValor → atualiza e causa novo render  
// 'inicial' → valor no primeiro render  
  
setValor('novo'); // → componente re-renderiza com "novo"
```

Renderização condicional

```
// Forma 1: ternário (se/senão)  
{error ? <Text>{error}</Text> : null}  
  
// Forma 2: curto-circuito (só renderiza se error for verdadeiro)  
{error && <Text>{error}</Text>}  
  
// Forma 3: array de estilos condicionais  
style={ [styles.base, error && styles.inputError] }  
// Se error for falsy (null, undefined, ''), inputError é ignorado
```

Optional Chaining (?.) e Nullish Coalescing (||)

```
const prev = route?.params?.dados || {};  
// route?.params → se route for null/undefined, retorna undefined (sem erro)  
// ?.dados → se params for null/undefined, retorna undefined  
// || {} → se o resultado for falsy, usa objeto vazio como fallback
```

Spread Operator (...)

```
// Em objetos: copia todas as propriedades
const payload = { ...dadosPasso1, endereco, planos: selecionados };
// Resultado: { nome, email, senha, repetirSenha, endereco: {...}, planos: [...] }

// Em arrays: cria novo array com os itens anteriores + novo
const novoArray = [...anterior, novoItem];
```

Array.map() para listas dinâmicas

```
{PLANOS.map((plano) => (
  <Checkbox
    key={plano}          // Obrigatório: identifica cada item da lista para o React
    label={plano}
    checked={selecionados.includes(plano)}
    onPress={() => toggle(plano)}
  />
))}
```

Flexbox no React Native

CSS Web

`flex-direction: row` (padrão web)

Aceita porcentagens e `px`

`display: flex` é opt-in

React Native

`flexDirection: 'column'` (padrão RN)

Só aceita números (density-independent pixels)

Todos os elementos já são flex por padrão

12 Diagrama geral da arquitetura



utils/masks.js → máscaras e validações

Resumo dos conceitos por arquivo

Conceito	Onde é usado
Componentes funcionais	Todos os arquivos .jsx
useState (estado local)	Todas as telas e componentes
Props e composição	Button, Input, Checkbox, Logo, ScreenContainer
React Navigation (Stack)	AppNavigator.jsx
Passagem de dados entre telas	route.params em todas as telas de cadastro
Validação de formulário	handleLogin, validar() em cada tela
Máscaras de input	Input.jsx + MaskInput (CEP e Telefone)
Regex para validação	isValidEmail, isValidCEP, isValidPhone
Renderização condicional	Erros, labels, spinner do botão
StyleSheet e Flexbox	Todos os arquivos
Tema centralizado	theme/colors.js usado por todos
SVG em React Native	Logo.jsx com react-native-svg
Safe Area / Keyboard Avoid	ScreenContainer.jsx



Próximos passos sugeridos para evoluir o projeto:

- Integrar com uma API real (fetch/axios) nos pontos marcados com comentários
- Adicionar AsyncStorage para manter o usuário logado
- Implementar Context API para gerenciar o estado do usuário autenticado
- Adicionar tratamento de erros de rede (try/catch, estados de loading)

- Criar uma tela
Home
para usuários logados